

CL2006 - Operating Systems Spring 2024

LAB # 6 MANUAL (Common)

Please note that all labs' topics including pre-lab, in-lab and post-lab exercises are part of the theory and labsyllabus. These topics will be part of your Midterms and Final Exams of lab and theory.

Objectives:

Demonstrate System Calls Programmatically:

1. Develop an understanding of system calls by implementing various functions programmatically and explore the functionality and usage of each system call through practical coding exercises, emphasizing their role in ordinary pipes, named pipes and shared memory.
2. Implement C program producing and consuming data using ordinary and names pipes and shared memory by utilizing appropriate system calls.

Lab Tasks:

Use of system calls related to the following:

1. Ordinary Pipes
2. Named Pipes
3. Shared Memory.

Delivery of Lab contents:

Strictly following the following content delivery strategy. Ask students to take notes during the lab.

1st Hour

- Experiment 6a

2nd Hour

- Experiment 6b

3rd Hour

- Experiment 6c

Initial Document: Hamza Yousuf (January 2019). This version by: **Nadeem Kafi (23/02/2024)**
DEPARTMENT OF COMPUTER SCEICEN, FAST-NU, KARACHI

**** ChatGPT is heavily used to make the contents of this document along with other Internet sources.**

EXPERIMENT 6a

InterProcess Communication using Pipes

OBJECTIVE:

- Learn and Understand InterProcess Communication using implementation of Pipes

Pipes:

Ordinary pipes allow two processes to communicate in standard producer consumer fashion: the producer writes to one end of the pipe (the **write-end**) and the consumer reads from the other end (the **read-end**). **As a result, ordinary pipes are unidirectional**, allowing only one-way communication. If two-way communication is required, two pipes must be used, with each pipe sending data in a different direction.

- A pipe has a read end and a write end.
- Data written to the write end of a pipe can be read from the read end of the pipe.

Creating an Ordinary Pipe:

```

1  #include <sys/types.h>
2  #include <stdio.h>
3  #include <string.h>
4  #include <unistd.h>
5  #define BUFFER_SIZE 25
6  #define READ_END 0
7  #define WRITE_END 1
8  int main(void) {
9      char write_msg[BUFFER_SIZE] = "Greetings";
10     char read_msg[BUFFER_SIZE];
11     int fd[2];
12     pid_t pid;
13
14     if (pipe(fd) == -1) { // create a pipe
15         fprintf(stderr, "Pipe failed");
16         return 1;
17     }
18     pid = fork();
19     if (pid < 0) { /* error occurred */
20         fprintf(stderr, "Fork Failed");
21         return 1;
22     }
23     if (pid > 0) { /* parent process */
24         close(fd[READ_END]);
25         write(fd[WRITE_END], write_msg, strlen(write_msg) + 1);
26         close(fd[WRITE_END]);
27     }
28     else { /* child process */
29         close(fd[WRITE_END]); // close the unused end of the pipe
30         read(fd[READ_END], read_msg, BUFFER_SIZE); // read from the pipe
31         printf("read %s", read_msg);
32         close(fd[READ_END]); // close the read end of the pipe
33     }
34     return 0;
35 }

```

In-Lab

- Compile and execute the above code. Open a separate terminal window to note the process ids using in this code.
- Modify the code such that it takes write_msg value from user terminal.

EXPERIMENT 6b

Named Pipes

OBJECTIVE:

- Learn and execute InterProcess Communication using implementation of named pipes.

Named Pipes:

- It is an extension to the traditional pipe concept on Unix. A traditional pipe is “unnamed” and lasts only as long as the process.
- A named pipe, however, can last as long as the system is up, beyond the life of the process. It can be deleted if no longer used.
- Usually, a named pipe appears as a file, and generally processes attach to it for inter-process communication. A FIFO file is a special kind of file on the local storage which allows two or more processes to communicate with each other by reading/writing to/from this file.
- A FIFO special file is entered into the filesystem by calling `mkfifo()` in C. Once we have created a FIFO special file in this way, any process can open it for reading or writing, in the same way as an ordinary file. However, it must be open at both ends simultaneously before you can proceed to do any input or output operations on it.
- Reading from or writing to a named pipe occurs just like traditional file reading and writing; except that the data for named pipe is never written to or read from a file in hard disk but memory.

Producer code using Named pipes:

```

8
9  #define FIFO_FILE "/tmp/myfifo"
10
11 int main() {
12     int fd;
13     char buffer[BUFSIZ];
14     ssize_t num_bytes;
15
16     mkfifo(FIFO_FILE, 0666);           // Create the named pipe (FIFO)
17     fd = open(FIFO_FILE, O_WRONLY);    // Open the named pipe for writing (producer)
18     if (fd == -1) {
19         perror("open");
20         exit(EXIT_FAILURE);
21     }
22     while (1) { // Producer loop
23         printf("Producer: Enter a message (or 'exit' to quit): ");
24         fgets(buffer, BUFSIZ, stdin);
25         num_bytes = write(fd, buffer, strlen(buffer)); // Write input to the named pipe
26         if (num_bytes == -1) {
27             perror("write");
28             exit(EXIT_FAILURE);
29         }
30         if (strncmp(buffer, "exit", 4) == 0) { // Check for exit condition
31             break;
32         }
33     }
34     close(fd);           // Close the named pipe
35     unlink(FIFO_FILE);   // Remove the named pipe from the file system
36
37     return 0;
38 }
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <sys/types.h>
6  #include <sys/stat.h>
7  #include <string.h>

```

In-Lab:

1. Compile and execute the producer code above that uses named pipe. Correct errors, if any.
2. Carefully study the producer code and verify that the FIFO_FILE has been created.
3. Now, write consumer code for the above producer using the following hints:
 - Open FIFO_FILE in read only mode using O_RDONLY
 - Consumer read the data using num_bytes = read(fd, buffer, BUFSIZ);
 - Display the data sent by the producer using
printf("Consumer: Received message: %s", buffer);

EXPERIMENT 6c

InterProcess Communication using Shared Memory

OBJECTIVE:

- Learn and Understand InterProcess Communication using implementation of Shared Memory

Shared Memory:

InterProcess Communication through shared memory is a concept where two or more processes can access the common memory. And communication is done via this shared memory where changes made by one process can be viewed by another process.

Producer process using POSIX shared memory API:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <fcntl.h>
5  #include <sys/shm.h>
6  #include <sys/stat.h>
7  #include <sys/mman.h>
8  #include <unistd.h>
9
10 int main() {
11     const int SIZE = 4096;           // the size (in bytes) of shared memory object
12     const char *name = "OS";         // name of the shared memory object
13     const char *message_0 = "Hello"; // strings written to shared memory
14     const char *message_1 = "World!";
15     int fd;                          // shared memory file descriptor
16     char *ptr;                       // pointer to shared memory object
17
18     fd = shm_open(name, O_CREAT | O_RDWR, 0666); // create the shared memory object
19     ftruncate(fd, SIZE); // configure the size of the shared memory object
20     // memory map the shared memory object
21     ptr = (char *)mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
22     // write to the shared memory object
23     sprintf(ptr, "%s", message_0);
24     ptr += strlen(message_0);
25     sprintf(ptr, "%s", message_1);
26     ptr += strlen(message_1);
27
28     return 0;
29 }
```

```
gcc test.c -o test -lrt
```

Consumer process using POSIX shared-memory API:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <fcntl.h>
4  #include <sys/shm.h>
5  #include <sys/stat.h>
6  #include <sys/mman.h>
7  int main() {
8      const int SIZE = 4096;
9      const char *name = "OS";
10     int fd;
11     char *ptr;
12     fd = shm_open(name, O_RDONLY, 0666);
13     ptr = (char *)mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
14     printf("%s", (char *)ptr); // read from the shared memory object
15     shm_unlink(name);         // remove the shared memory object
16     return 0;
17 }

```

In-lab

- Compile and execute producer code.
- Compile and execute the consumer code. It will give a run-time error. Why?
- Now modify the producer code such that the producer and the consumer code run concurrently as two processes. Now the consumer code will print the message.
- Now modify the consumer code to print the second message.

------(X)-----